Multi – level linked list

–Insert At Beginning – Program Analysis and Time Complexity

Program:

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}
// --- 1. Insert at Beginning (Main Level) ---
void insertAtBeginning()
{
    int data;
    cout << "Enter data: ";
    cin >> data;

    Node *newNode = createNode(data);
    newNode->next = head;
    head = newNode;
    cout << "Inserted " << data << " at beginning." << endl;
}
……
        cout << "1. Insert at Beginning (Main)" << endl;
       case 1:
            insertAtBeginning();
            break;
```

Program Analysis

Say, int data = 20 .

Now, we pass this data to function:

Node * newNode = createNode(data: 20);

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}
```

Next,

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 −bit systems, regardless of the size of the structure it points to.

→ ***The size of the Node structure itself is determined by the sum of the sizes of its members***:

→ $data$(***an integer***) ***is typically*** 4 ***bytes***.

→ $next$(***a pointer pointing to another Node***) ***is typically*** 8 ***bytes***.

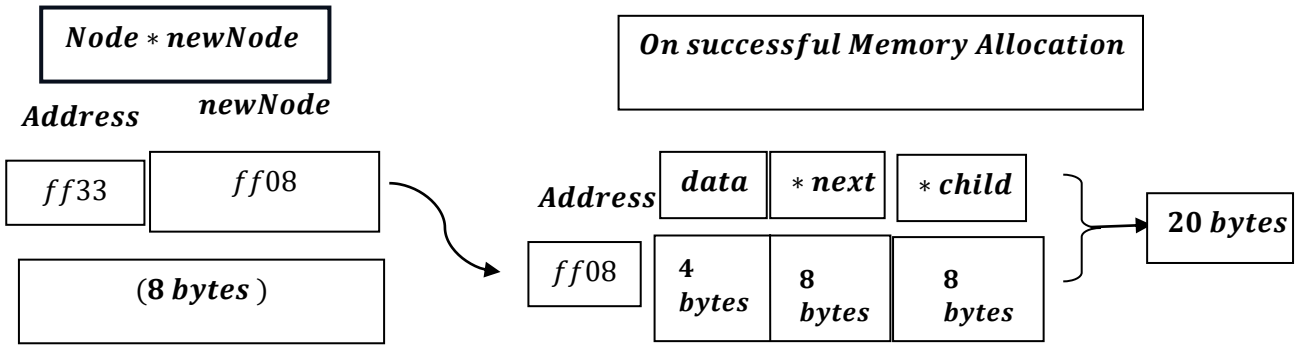
→ $child$(***a pointer pointing to child node***)***is typically*** 8 ***bytes***.

→ ***Therefore*** , ***the size of the Node structure is typically*** 20 ***bytes***(4 ***bytes for data*** $+ 8$ ***bytes for next*** $+ 8$ ***bytes for child***) .

What does it mean?

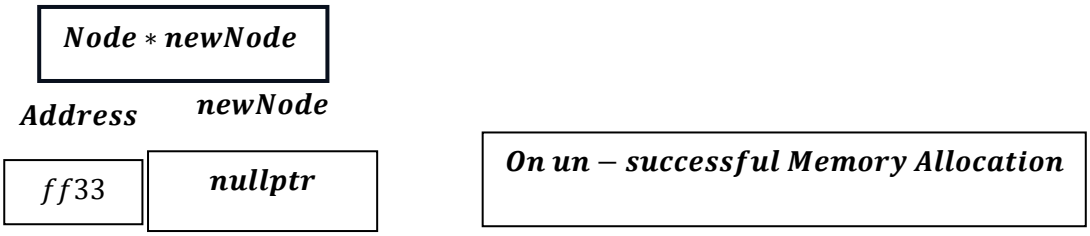
→ ***newNode will try to allocate memory for integer data for*** 4 ***bytes and next pointer variable i.e. typically*** 8 ***bytes, also child next pointer variable also typically*** 8 ***bytes hence assumed total space consumed is total*** $(4 + 8 + 8)$***bytes*** $= 20$ ***bytes***.

Say,



***But on Un – Successful memory allocation*:**

→ if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.



***Now, if the value of the pointer variable newNode is nullptr, then*:**

One may output : `Overflow ***or Memory Allocation Failed as a message.***`

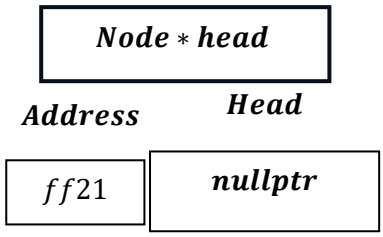
Now,

```
        cout << "1. Insert at Beginning (Main)" << endl;
    case 1:
        insertAtBeginning();
        break;
```

Now , this will mainly create Main node not a child node.

How?

```
Node *head =nullptr
```



And our data input is : 20.

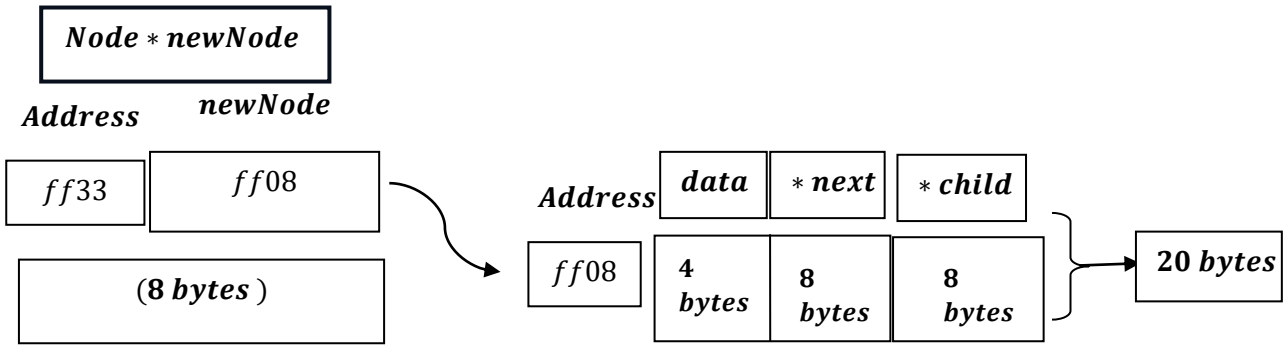
```
Node *createNode(int data){…}
```

Return type is struct type pointer variable as mentioned here:

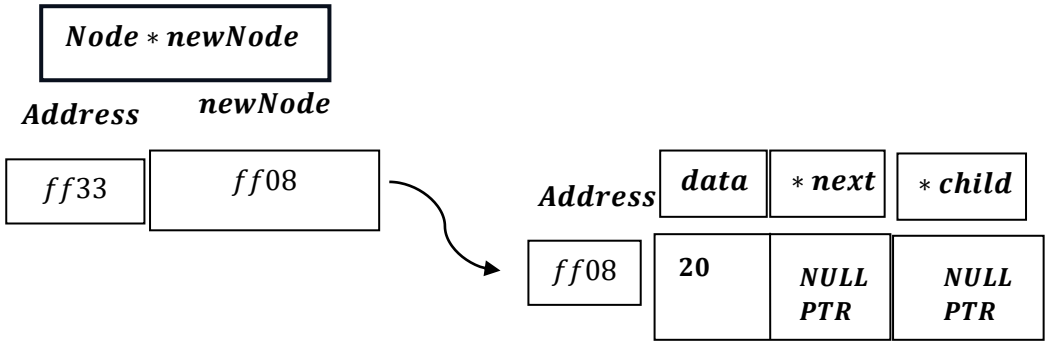
Node * and function name is createNode(variable : 20).

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

We got:



```
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
```




```
    return newNode;
```

As we already know , Return type is struct type pointer variable . Return newNode will return the value stored inside newNode, pointer :

i.e. return newNode : ff08 .

Now,

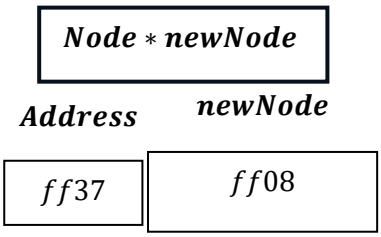
```
    Node *newNode = createNode(data);
```

Node * newNode = ff08.

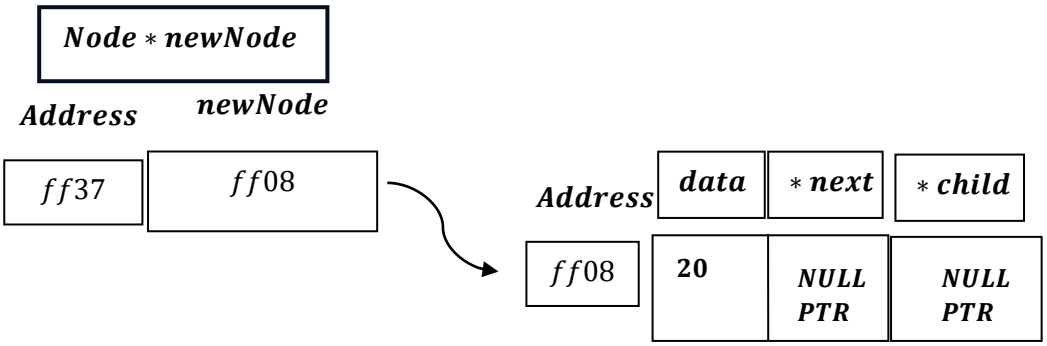
This newNode is another pointer variable , under scope of insertAtBeginning() function, not to be mixed with createNode() function .

For * newNode of insertAtBeginning() function:

Node * newNode = ff08.

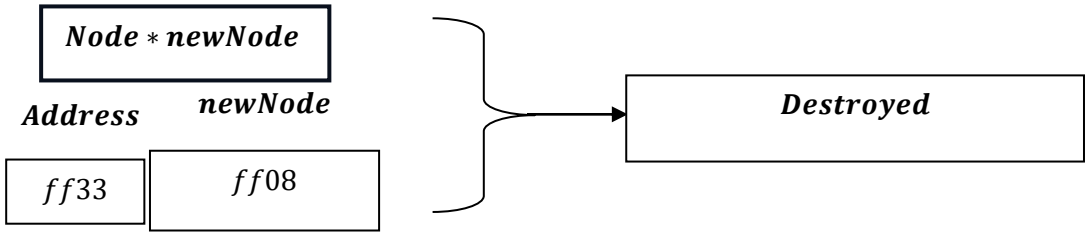


Now pointer * newNode connecting to address ff08.

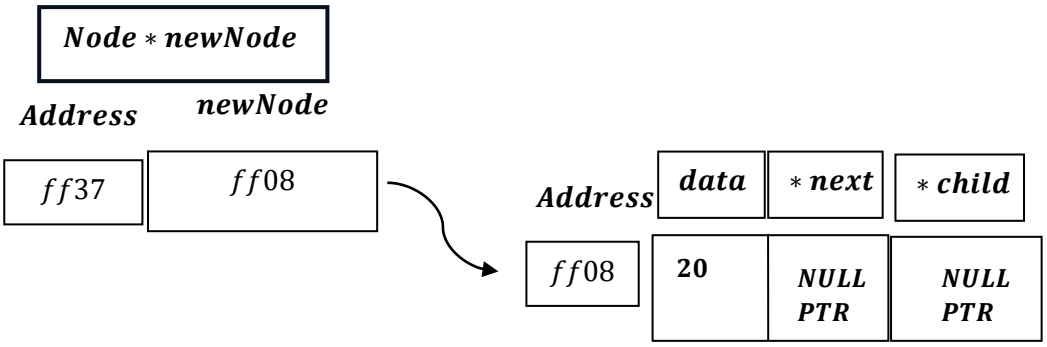



```
    newNode->next = head;
```

Notably, after function createNode() finishes , the local variable of the function gets destroyed:



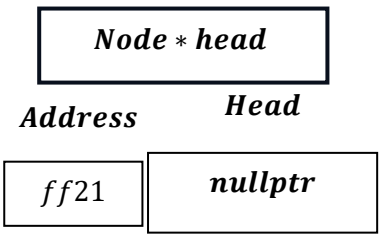
Hence now we have , pointer variable newNode of insertAtBeginning() :




```
    newNode->next = head;
```

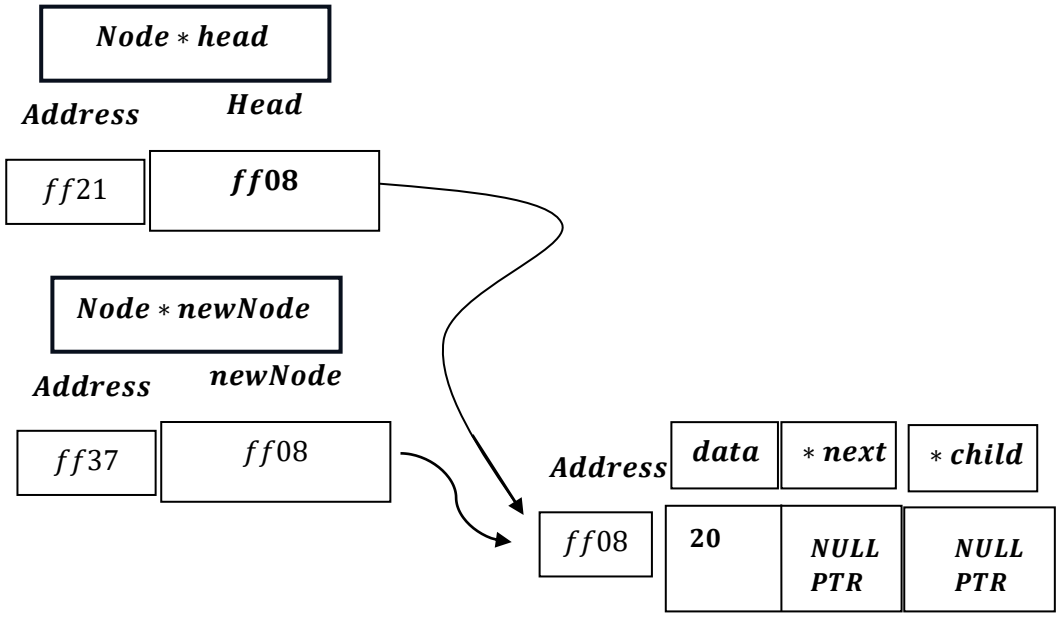
Here, for now , newNode → next = Head = NULLPTR.

As , pointer variable * Head = NULLPTR.



```
    head = newNode;
```

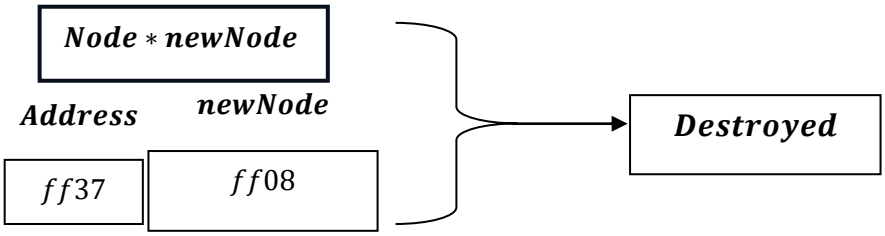
$Head = newNode = ff08$.



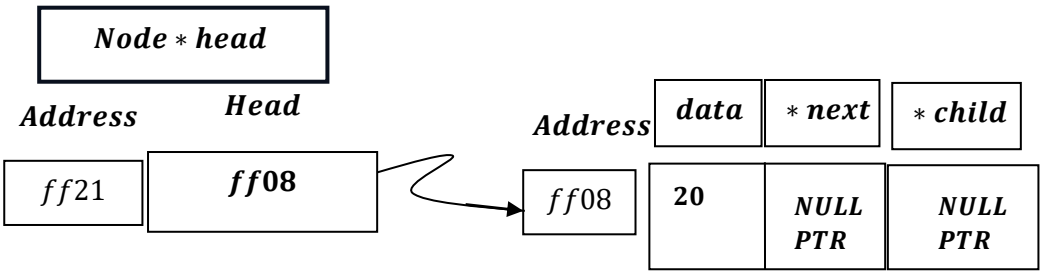
Now, it will print:

Inserted data : 20 at beginning.

And , Node * newNode of insertAtBeginning(), gets destroyed after execution of the function:



And we are left with:

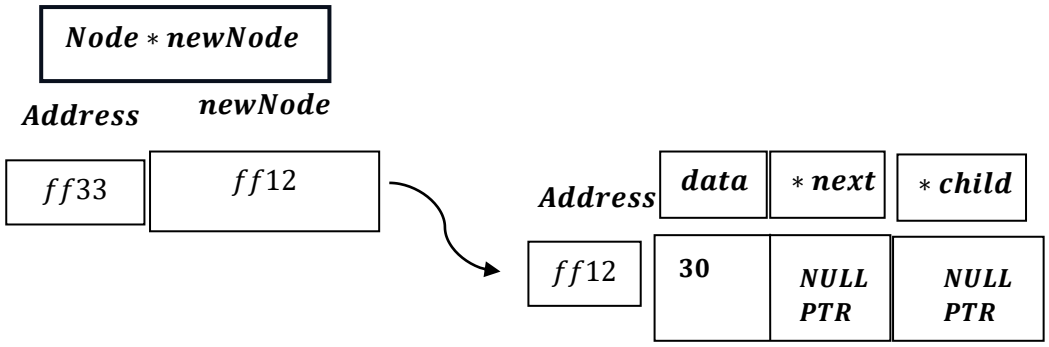


If we enter another data say : **30**.

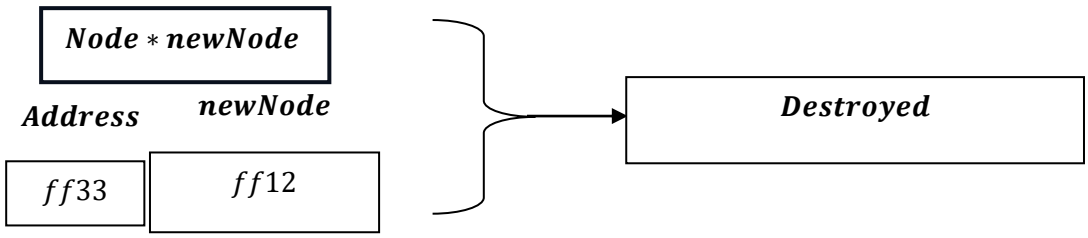
```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}
void insertAtBeginning()
{
    int data;
    cout << "Enter data: ";
    cin >> data;

    Node *newNode = createNode(data);
    newNode->next = head;
    head = newNode;
    cout << "Inserted " << data << " at beginning." << endl;
}
```

1)*CreateNode Function*:

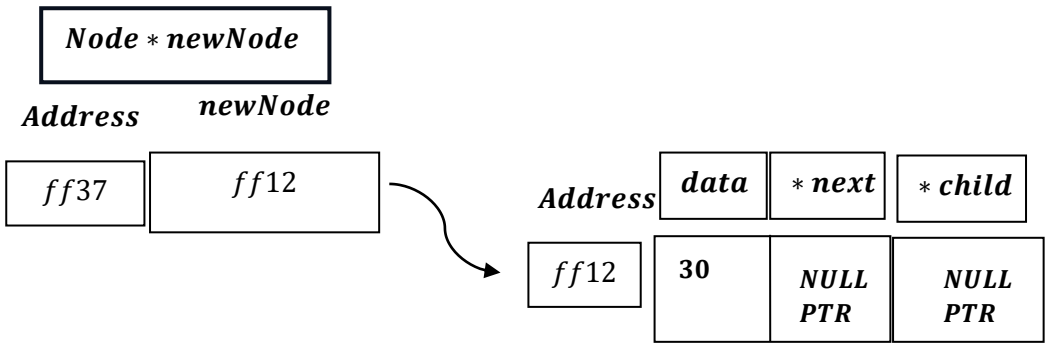


As function ends :

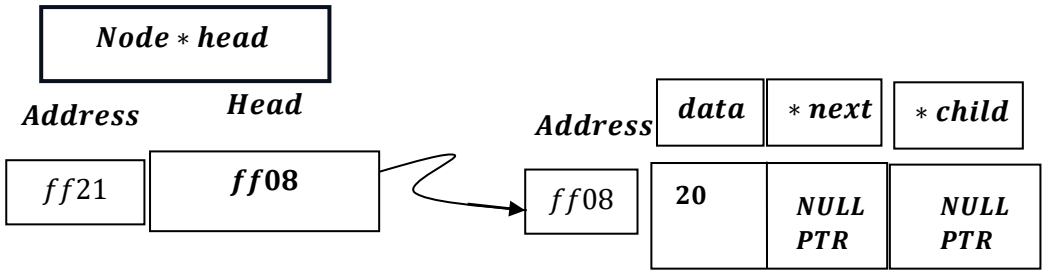


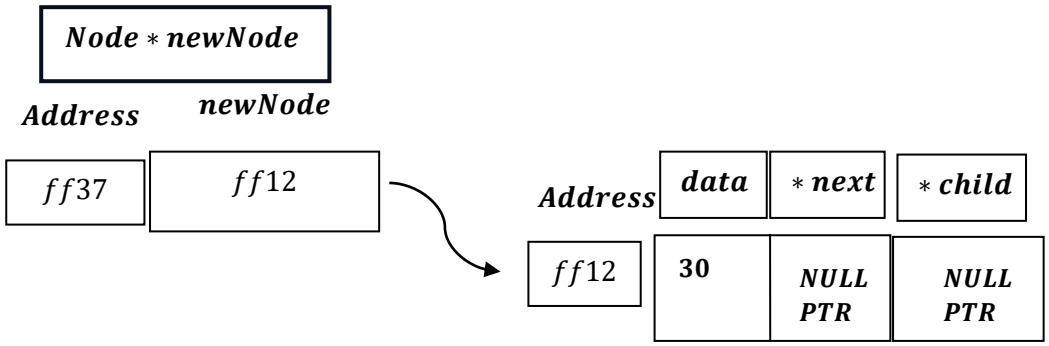
2)*Insert At Beginning Function*:

```
    Node *newNode = createNode(data);
```

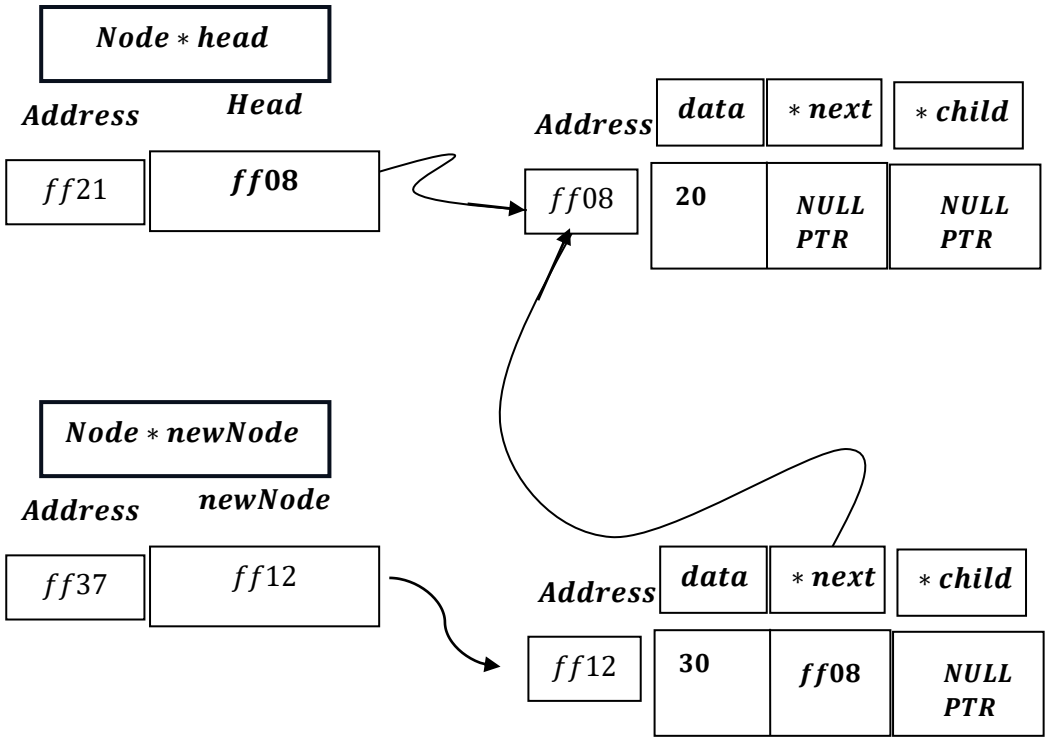


```
    newNode->next = head;
```



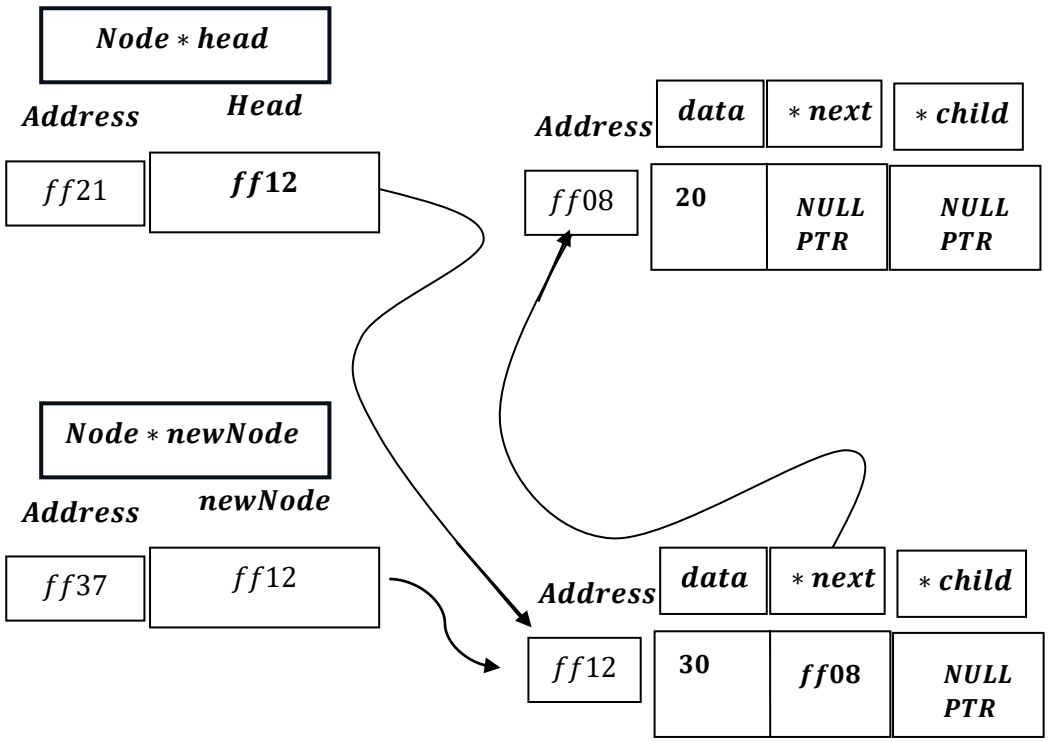


$$newNode \rightarrow next = head = ff08\,;$$

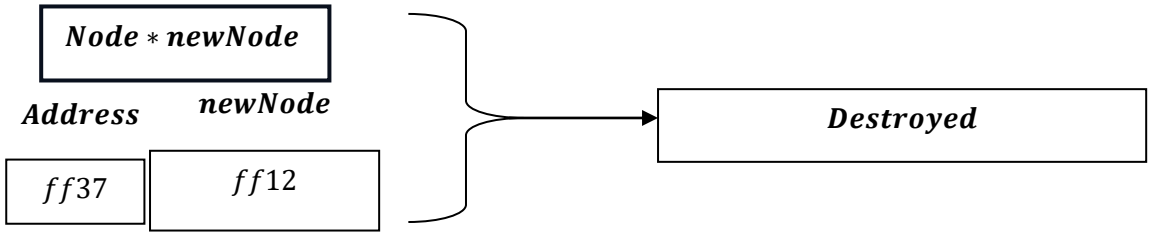



```
    head = newNode;
```

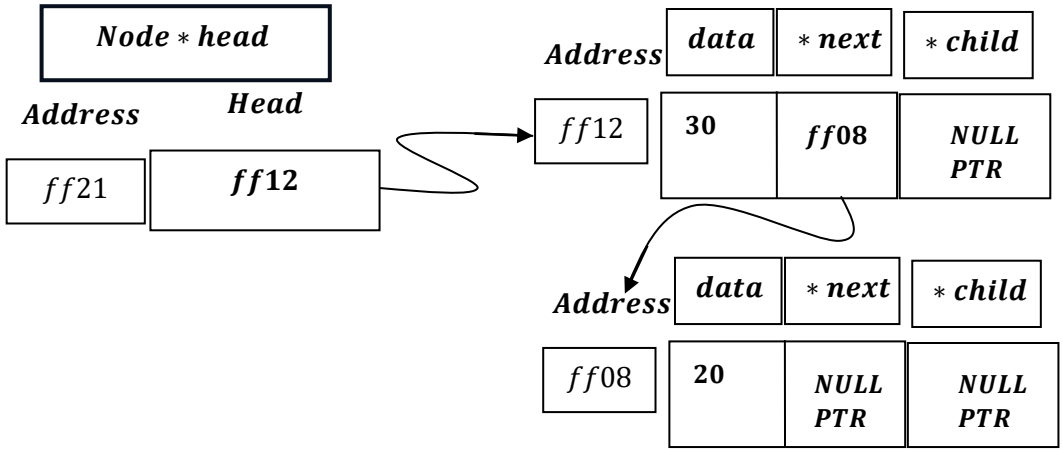
$head = newNode = ff12$



Now after function : insertAtBeginning() finished ,



And , we are left with:



$\boldsymbol{Time\ Complexity: O(1)\ (Constant\ Time)}$

$Because$:

1. $createNode(data)$
   - $Allocates\ memory\ for\ exactly\ one\ node.$
   - $Assigns\ values\ to\ a\ few\ fields.$
   - $No\ loops\ or\ recursion.$
   - $Constant\ operations.$
2. $newNode{-}{>}\ next\ =\ head;$
   - $Single\ pointer\ assignment.$
3. $head\ =\ newNode;$
   - $Single\ pointer\ update.$

$All\ operations\ take\ constant\ time\ regardless\ of\ list\ size\ n.$

$\boldsymbol{So}$:

$$\boldsymbol{T(n) = c}$$

$\boldsymbol{Hence}$:

$$\boldsymbol{O(1)}$$

$$\boldsymbol{Hence\ ,Best\ Case = Worst\ Case = Average\ Case = \Omega(1) = O(1) = \Theta(1).}$$

Multi − level Linked List	***Cases***	***Time Complexity***
InsertAt Beginning	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(1)}$
	3. ***Average Case***	$\boldsymbol{\Theta(1)}$

****
